

CasaSwap

Documentación Técnica del Proyecto

Plataforma de intercambio de viviendas en España
mediante un sistema de puntos

Proyecto Intermodular (PiM) · Desarrollo de Aplicaciones Web

Aplicación desplegada: <https://casa-swap.vercel.app>

Índice

1. Introducción y objetivos
2. Tecnologías utilizadas
3. Arquitectura de la aplicación
4. Diseño de la base de datos
5. Funcionalidades por tipo de usuario
6. El sistema de puntos
7. La API (backend)
8. Estructura del proyecto
9. Despliegue en producción
10. Enlaces y credenciales
11. Conclusiones y mejoras futuras

1 · Introducción y objetivos

CasaSwap es una aplicación web full-stack que permite a particulares **intercambiar sus viviendas** para sus vacaciones o escapadas sin que medie dinero real. En lugar de pagar con euros, la plataforma utiliza un sistema de **puntos** como moneda interna: cada usuario gana puntos cuando otra persona alquila su casa, y los gasta para alojarse en las casas de los demás.

La idea nace de plataformas reales de intercambio de casas, adaptada a un modelo gamificado mediante puntos que equilibra el valor de cada vivienda según sus características.

Objetivos del proyecto

- Desarrollar una aplicación web completa (frontend + backend + base de datos).
- Implementar autenticación de usuarios y un panel de administración.
- Diseñar una lógica de negocio propia: el cálculo y la transferencia de puntos.
- Integrar servicios externos en la nube (almacenamiento de imágenes y mapas).
- Desplegar la aplicación en producción y dejarla accesible públicamente.

2 · Tecnologías utilizadas

Capa	Tecnología	Uso
Frontend	Angular 21 (TypeScript)	Interfaz de usuario (SPA) con componentes standalone y signals
Estilos	CSS3	Diseño responsive, animaciones y tema visual propio
Backend	PHP 8.2 + PDO	API REST que gestiona la lógica y el acceso a datos
Base de datos	MySQL 8	Almacenamiento de usuarios, casas y solicitudes
Mapas	Leaflet + OpenStreetMap	Mapa interactivo en la ficha de cada casa
Imágenes	Cloudinary	Almacenamiento de las fotos en la nube
Hosting frontend	Vercel	Despliegue del frontend Angular
Hosting backend/BD	Railway	Despliegue del backend PHP y la base de datos MySQL
Contenedores	Docker	Imagen del backend para Railway
Control de versiones	Git + GitHub	Repositorio y despliegue continuo

3 · Arquitectura de la aplicación

La aplicación sigue una arquitectura cliente-servidor desacoplada en tres servicios en la nube que se comunican entre sí:

```
Navegador del usuario
|
|--> Vercel ..... Frontend Angular (SPA)
|
|--> Railway ..... API PHP + base de datos MySQL
|--> Cloudinary ... almacenamiento de fotos
```

El **frontend** (Angular) se ejecuta en el navegador y realiza peticiones HTTP (JSON) a la **API PHP**, que a su vez consulta la **base de datos MySQL**. Las **imágenes** no se guardan en el servidor, sino que se suben directamente desde el navegador a **Cloudinary**, que devuelve una URL que se almacena en la base de datos. Este desacople permite que cada parte escale de forma independiente y evita problemas de almacenamiento en servidores efímeros.

4 · Diseño de la base de datos

La base de datos **casaswap** consta de tres tablas relacionadas:

Tabla: usuarios

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
nombre, apellidos	VARCHAR	Datos personales
email	VARCHAR (único)	Correo / login
telefono, ciudad, provincia	VARCHAR	Datos de contacto
puntos	INT	Saldo de puntos (50 al registrarse)
password	VARCHAR	Contraseña
fecha_registro	DATE	Fecha de alta

Tabla: casas

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
usuario_id	INT (FK)	Propietario de la casa
titulo, descripcion	VARCHAR/TEXT	Información del anuncio
direccion, ciudad, barrio, provincia	VARCHAR	Ubicación
tipo_vivienda	ENUM	piso / casa / chalet / apartamento
num_habitaciones, capacidad	INT	Características
acepta_mascotas, tiene_piscina, tiene_patio	TINYINT	Servicios (0/1)
fecha_disponible_inicio / fin	DATE	Periodo disponible
puntos_base, valor_puntos	INT	Coste en puntos (auto + ajuste)
disponible	TINYINT	Si está disponible (0/1)
imagen_url, fotos	VARCHAR/TEXT	Portada y array JSON de fotos
fecha_publicacion	DATE	Fecha de publicación

Tabla: solicitudes

Campo	Tipo	Descripción
id	INT (PK)	Identificador único
casa_id, inquilino_id, propietario_id	INT (FK)	Relaciones de la solicitud
puntos	INT	Coste congelado al solicitar
fecha_inicio, fecha_fin	DATE	Fechas solicitadas
mensaje	TEXT	Mensaje al propietario
estado	ENUM	pendiente / aceptada / rechazada / cancelada
fecha_solicitud, fecha_resolucion	DATETIME	Marcas de tiempo

5 · Funcionalidades por tipo de usuario

Visitante (sin registro)

- Explorar el catálogo de casas con filtros (provincia, tipo, fechas).
- Ver la ficha completa de cada vivienda (fotos, mapa, servicios).

Usuario registrado

- Registrarse e iniciar sesión.
- Publicar, editar y eliminar sus propias casas (con subida de fotos).
- Gestionar su saldo de puntos.
- Solicitar el alquiler de otras casas indicando fechas y mensaje.
- Aceptar o rechazar las solicitudes que recibe sobre sus casas.
- Consultar el estado de sus solicitudes enviadas y cancelarlas.

Administrador

- Listar, editar y eliminar cualquier usuario.
- Listar, editar y eliminar cualquier casa.

6 · El sistema de puntos

Es la lógica de negocio central del proyecto. Cada casa tiene un valor en puntos calculado automáticamente a partir de sus características:

Concepto	Puntos
Tipo: piso / apartamento / casa / chalet	20 / 25 / 35 / 50
Por cada habitación	+8
Por cada persona de capacidad	+4
Piscina	+30
Patio o jardín	+15
Admite mascotas	+5
Ciudad premium (Madrid, Barcelona, Sevilla, Bilbao, Valencia, Málaga, San Sebastián)	+20

El propietario puede **ajustar manualmente** el precio $\pm 30\%$ sobre el valor automático. El cálculo se realiza tanto en el frontend (vista previa) como en el backend (autoritativo), garantizando coherencia.

Flujo de una transacción (con aprobación)

- 1 El usuario solicita una casa: se crea una solicitud en estado **pendiente** (sin mover puntos).
- 2 El propietario ve la solicitud en su perfil y la **acepta o rechaza**.
- 3 Al aceptar, se ejecuta una transacción atómica: se restan los puntos al inquilino y se suman al propietario, la casa pasa a no disponible y se rechazan las demás solicitudes pendientes de esa casa.

7 · La API (backend)

El backend es una API REST sencilla en PHP. Toda la comunicación se hace por **POST** a `servicios.php` enviando un JSON con el campo `accion`. Principales acciones disponibles:

Acción	Función
LoginUsuario / Login	Autenticación de usuario / administrador
ListarCasasDisponibles	Catálogo público de casas
ObtenerCasald	Ficha completa de una casa
AnadeCasa / ModificaCasa / BorraCasa	Gestión de casas
ListarCasasUsuario	Casas de un propietario
ObtenerPuntos	Saldo de puntos de un usuario
CrearSolicitud	Solicitar el alquiler de una casa
ListarSolicitudesRecibidas / Enviadas	Bandeja de solicitudes
AceptarSolicitud / RechazarSolicitud / CancelarSolicitud	Gestión de solicitudes
ListarUsuarios / ObtenerUsuariold / ...	Gestión de usuarios (admin)

8 · Estructura del proyecto

```
CasaSwap/      Frontend Angular
src/app/
  components/  portada, explorar, detalle-casa, perfil, ...
  services/    api.service, auth.service, toast.service
  models/      casa, usuario, solicitud
  guards/      auth.guard (protección de rutas)
  environments/ configuración por entorno (dev / prod)
backend/      API PHP
  servicios.php enrutador de acciones
  modelos.php   acceso a datos (PDO)
  Dockerfile    imagen para Railway
BD/           Scripts SQL de la base de datos
```

9 · Despliegue en producción

La aplicación está desplegada y accesible públicamente. El despliegue es continuo: cada cambio subido a la rama main de GitHub se publica automáticamente.

Servicio	Plataforma	Función
Frontend	Vercel	Sirve la SPA de Angular compilada
Backend	Railway (Docker)	API PHP con servidor integrado
Base de datos	Railway (MySQL)	Datos de la aplicación
Imágenes	Cloudinary	Almacenamiento de fotos (subida sin firma)

La conexión a la base de datos y las URLs de los servicios se configuran mediante **variables de entorno** (backend) y archivos de entorno de Angular (frontend), de modo que el mismo código funciona en local (XAMPP) y en producción.

10 · Enlaces y credenciales

Recurso	Enlace
Aplicación (web)	https://casa-swap.vercel.app
Repositorio (GitHub)	https://github.com/lucasfdezp/CasaSwap
API (backend)	https://casaswap-production.up.railway.app

Credenciales de prueba

Rol	Usuario	Contraseña
Administrador	admin@casaswap.es	admin123
Usuario	carlos@ejemplo.com	1234
Usuario	maria@ejemplo.com	1234
Usuario	pedro@ejemplo.com	1234
Usuario	lucia@ejemplo.com	1234
Usuario (demo)	demo@casaswap.com	1234

11 · Conclusiones y mejoras futuras

El proyecto cumple los objetivos planteados: una aplicación web completa, funcional y desplegada en producción, con una lógica de negocio propia (el sistema de puntos) y la integración de varios servicios en la nube. Durante su desarrollo se han trabajado el frontend (Angular), el backend (PHP), el diseño de bases de datos relacionales y el despliegue real en la nube.

Posibles mejoras futuras

- Cifrado de contraseñas (hash) en lugar de texto plano.
- Sistema de valoraciones y reseñas entre usuarios.
- Notificaciones por email al recibir o aceptar una solicitud.
- Chat interno entre inquilino y propietario.

- Historial de intercambios realizados y devolución de la casa a 'disponible'.